

Integrity and security

- ⊗ Integrity, Importance of integrity
- ⊗ Integrity constraints, Types: Domain
Referential
Trigger
Assertion
- ⊗ Integrity violation
- ⊗ security
- ⊗ Different security level
- ⊗ Authorization, Types
- ⊗ Authorization and views
- ⊗ Authorization Grant Graph
- ⊗ Revoking, Limitationsⁱⁿ authorization
- ⊗ Privileges
- ⊗ Role
- ⊗ Encryption
- ⊗ Decryption
- ⊗ Authentication
- ⊗ Audit trails

Integrity: Data integrity is a fundamental component of information security. In this broadcast we use 'data integrity' refers to the accuracy and consistency of data stored in database. Data with integrity is said to have a complete on whole structure.

As a example, To maintain data integrity numeric column/ cells should not accept alphabetic data.

Importance of integrity:

- ⇒ Using stored procedures to control access to data.
- ⇒ Enforcing business rules in the code of database application.
- ⇒ Enforcing business rules with triggered stored database procedure.

Integrity constraints: Integrity constraints can be an arbitrary predicate pertaining to the database. Integrity constraints ensure that changes made to the database by authorized users do not result in a loss of data consistency.

Example of integrity constraints are:

- ⇒ An instructor name cannot be null.
- ⇒ No two instructors can have the same instructor ID.

Integrity constraint are usually identified as part of the database schema design process and declared as part of the create table command used to create relation on

Types of integrity constraints: There are 4 types of integrity constraints.

- ① Domain Constraint
- ② Referential integrity constraints
- ③ Trigger
- ④ Assertion

Domain constraints (column): A domain is defined as the set of all unique values permitted for an attribute.

Domain constraints are the most elementary form of integrity constraint. Domain constraints are user defined data types and ~~domain~~ we can define them like this:

Domain constraints = data type constraints

where constraints are not null, unique, Primary key, Foreign key, check, default.

⇒ New domains can be created from existing data types

Create domain Dollars numeric (12,2)

Create domain Pounds numeric (12,2)

⇒ We cannot assign or compare a value of type Dollars to a value of type Pounds.

Domain constraints not only allow us to test values inserted in the database but also permits us to test queries to ensure that the comparisons make sense.

Example - 01

To create a table 'student-Info' with "stu-id" field having value greater than 100, we can create a domain and table like this:

```
Create domain id-Value int
```

```
Constraint id-test
```

```
check (value > 100);
```

```
create table student, Info (
```

```
stu_id id-Value PRIMARY KEY,
```

```
stu_name Varchar (100),
```

```
stu_age int );
```

Referential integrity constraints: (Table)

Referential integrity refers to the relationship between tables.

This is concerned with the concept of foreign keys. The rule states that the foreign key value can be in two states. The 1st state is that the foreign key value would refer to a primary key value of another table or it can be null.

Being null could simply mean that there is no relationship or the relationship is unknown.

Formal definition:

- Let $r_1(R_1)$ and $r_2(R_2)$ be relations with primary keys k_1 and k_2 respectively.
- The subset α of R_2 is a foreign key referencing k_1 in relation r_1 . if for every t_2 in r_2 there must be a tuple t_1 in r_1 such that $t_1[k_1] = t_2[\alpha]$
- Referential integrity constraint also called subset dependency since it can be written as
$$\pi_{\alpha}(r_2) \subseteq \pi_{k_1}(r_1)$$

Insert: If a tuple t_2 is inserted into r_2 the system must ensure that there is a tuple t_1 in r_1 such that $t_1[k_1] = t_2[\alpha]$. That is,

$$t_2[\alpha] \in \pi_{k_1}(r_1)$$

During an insert operation, if a row is to be added to a table and that row contains a value in a foreign key column that is not present in the table referred by the foreign key, adding the row would violate referential integrity.

So, in this case, we must test the foreign key column value whether it is present in the referenced table or not.

Delete: If a tuple t_1 is deleted from r_1 , the system must compute the set of tuples in r_2 that reference t_1 .

$$\sigma_\alpha = t_1[k] (r_2)$$

During a delete operation if a row to be deleted contains a value that is referenced by a foreign key in another table, then the referential integrity can be avoided by two ways. They are:

1. Deleting the original row and also deleting the referencing row from the other table. This action is called cascaded deletion and can cascade through a series of referencing table.
2. Deleting neither of the rows i.e. taking no action. This action is called Restricted deletion.

$$\sigma_\alpha = t_1[k] (r_2)$$

If this set is not empty,

- ⇒ either the delete command is rejected as an error
- ⇒ the tuples that reference t_1 must themselves be deleted. (cascading deletions are possible).

update: There are two cases:

⇒ If a tuple t_2 is updated in relation r_2 and the update modifies values for foreign key α , then a test similar to the insert case is made:

Let t_2 denote the new value of t_2 .

$$t_2'[\alpha] = \pi_k(h_2)$$

⇒ If a tuple t_1 is updated in h_1 and the update modified values for the primary key(k) then a test similar to the delete case is made:

- The system must compute

$$G_\alpha = t_1[k](h_2)$$

- If this set is not empty

⇒ ^{The} update maybe rejected as an error on

⇒ The update maybe cascaded to the tuples in the set on,

⇒ The tuples in the set may be deleted.

Referential integrity in the E-R model:



Relationship set between entity sets E_1 and E_2 . The relational schema for R includes the primary keys k_1 of E_1 and k_2 of E_2 .

Then, k_1 and k_2 form foreign keys on the relational schemas for E_1 and E_2 respectively.

Weak entity sets are also a source of referential integrity constraints.

For the relation schema for a weak entity set must include the primary key attributes of the entity set on which it depends.

Referential integrity in SQL: Primary and candidate keys and foreign keys can be specified as part of the SQL create table statement.

- The primary key clause lists attributes that comprise the primary key.
- The unique key clause lists attributes that comprise a candidate key.
- The foreign key clause lists the attributes that comprise the foreign key and the name of the relation referenced by the foreign key.

foreign key (account-number) references account

Short form for specifying a single column as foreign key
account-number char(10) references account

Reference columns in the referenced table can be explicitly specified but must be declared as primary/candidate keys

foreign key (account-number) references account (account-number)

Create table customer

(customer-name char(20),

customer-street char(30),

customer-city char(30),

Primary key (customer-name))

Create table branch

(branch-name char(15),

branch-city char(30),

assets integer,

Primary key (branch-name))

Create table account

(account-number char(10),

branch-name char(15),

balance integer

Primary key (account-number),

foreign key (branch-name) references branch)

Create table depositon

(customer-name char(20),

account-number char(10),

Primary key (customer-name, account-number),

foreign key (account-number) references account,

foreign key (customer-name) references customer)

⇒ Alternative to cascading:

- on delete set null
- on delete set default

⇒ Null values in foreign key attribute complicate SQL referential integrity semantics and are the best prevented using not null.

- If any attribute of a foreign key is null, the tuple is defined to satisfy the foreign key constraint.

Assertion: overall database

An assertion is a predicate expressing a condition that we wish the database always to satisfy.

An assertion is a statement in SQL that ensures a certain condition will always exist in database.

⇒ Assertion are like column and table constraints except they are specified separately from table definitions

⇒ An assertion in SQL takes the form

create assertion <assertion-name> check <predicate>

⇒ when assertion is made the system tests it for validity and tests it again on every update that may violate the assertion.

⇒ Assertion for all x , $P(x)$
is achieved in a round about fashion. using
not exists x such that not $P(x)$

Example-1

The sum of all loan amounts for each branch must be less than the sum of all account balances at the branch.

Create assertion sum-constraint check

```
(not exists (select* from branch
             where (select sum(amount) from loan
                    where loan.branch-name =
                          branch.branch-name) >=
                    (select sum (amount) from account
                     where loan.branch-name =
                          branch.branch-name)))
```

Example-2

Every loan has at least one borrower who maintains an account with a minimum balance of \$1000.00

Create assertion balance-constraint check

(not exists (

select* from loan

where not exists (

select*

from borrower, depositor, account

where loan.loan-number = borrower.loan-number

and borrower.customer-name = depositor.

customer-name

and depositor.account-number = account.

account-number

and account.balance >= 1000)))

Triggers: A trigger is a statement that is executed automatically by the system as a side effect of a modification to the database.

Triggers are blocks of SQL code which are executed as a result of any insert/delete/update on the table.

There will not be any explicit trigger calls. It is always automatically executed. It will always be written as an SQL block.

The set of transactions inside the trigger can be performed on the same table or one or more than other tables.

To design a trigger mechanism, we must:

⇒ specify the condition under which the trigger is to be executed.

⇒ specify the actions to be taken when the trigger executes.

Necessity:

⇒ Triggers are easy to maintain and they enforce faster application development.

⇒ Triggers are useful for tasks such as enforcing business rules, validating input data and keeping an audit trail.

☐ Triggering ^{event} can be insert, delete on update

☐ Triggers on update can be restricted to specific attributes.

- Create trigger overdraft-trigger, after update of balance on account.

☐ Values of attributes before and after an update can be referenced.

⇒ Referencing old row as: For deletes and updates

⇒ Referencing new row as: For inserts and updates

☐ Triggers can be activated before an event, which can serve as extra constraints.

Eg. Convert blanks to null.

Create trigger setnull-trigger before update on h

referencing new row as nrow

for each row

when nrow.phone-number = ''

set nrow.phone-number = null

Statement level triggers:

- Use for each statement instead of for each row.
- use referencing old table or referencing new table to refer to temporary tables (called transition tables) containing the affected rows.
- Can be more efficient when dealing with SQL statements that update a large number of rows.
- Can be more efficient when dealing with SQL statements that update a large number of rows.

When not to use triggers:

- Maintaining summary data, Replicating database by recording changes to special relations (called change or delta relations)
- Databases provide built-in support for replication
- Encapsulation facilities can be used
 - ⇒ Define methods to update fields

Example:

```
create trigger overdraft_trigger after update on account
referencing new row as nrow
for each row
when nrow.balance < 0
begin atomic
    insert into borrowern
    (select customer-name, account-number
```


from depositor

where $\text{now.account-number} = \text{depositor.account-number}$;

insert into loan values

($\text{now.account-number}$, now.branch-name , - now.balance);

update account set $\text{balance} = 0$

where $\text{account.account-number} = \text{now.account-number}$

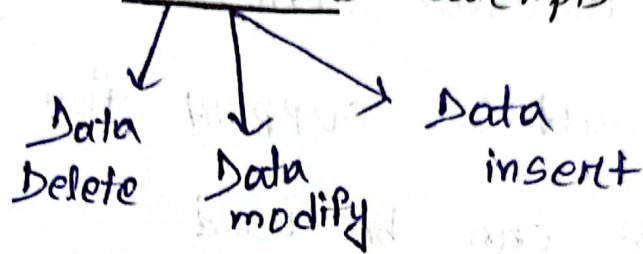
end.

Integrity violation: It occurs when an insert, update or delete statement violates a primary key, foreign key, check or unique constraint or a unique index.

Attempt to insert duplicate key row in object
<object-name> with unique index <index-name>.

Security: It is a safe guarding from malicious access by intruders.

Protection from malicious attempts to steal or modify data.



why do we need ^{security} in database?

- Database ~~user~~ security refers to the collective measures used to protect and secure a database.
- Databases often hold the backbone of an organization, its transactions, customers etc.
- It is an essential part of any company. It prevents the compromise or loss of data contain in the database.

Different security level:

① Database system level:

⇒ Authentication and authorization mechanisms to allow specific users access only to required data.

⇒ ~~if~~ some database system users maybe authorized to access only a limited portion of the database. Other users maybe allowed to issue queries but may be forbidden to modify the data.

It is responsibility of database system to ensure that these authorization restrictions are not violated.

② Operating system level:

operating system super-user can do anything they want to the database. Good operating system level security is required.

- Protection from invalid logins
- Protection from improper use of superuser authority
- File level access protection

③ Network level: Must use encryption to prevent

⇒ Eavesdropping (unauthorized reading of messages)

⇒ Masquerading (pretending to be an authorized user or sending messages supposedly from authorized user)

The network software is as important as physical security. Both on the internet and in networks private to an enterprise.

- Each site must ensure that it communicate with trusted sites.
- Link must be protected from theft of messages.
- Identification protocol (password-based)

④ Physical level: The sites containing the computer system must be secured against armed or hidden entry by intruders.

⇒ Physical access to computers allows destruction of data by intruders; traditional lock and-key security is needed. ⇒ Protection of network and terminal cables from wiretaps

⇒ Computers must be protected from floods, fire etc.

⇒ Protection of disks from theft, erasure, physical damage etc.

⑤ Human level:

⇒ Users must be screened to ensure that an authorized users do not give access to intruders.

⇒ Protection from stolen passwords, sabotage etc.

⇒ Users should be trained on password selection and secrecy.

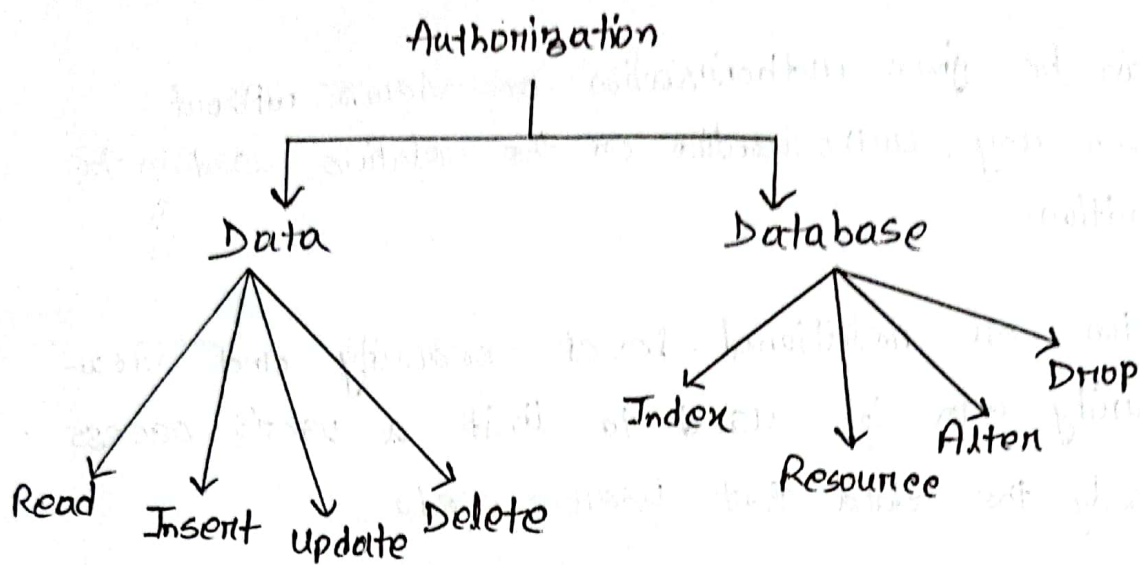
• Primary management problem:

1. Frequent change of password
2. Careful hiring practices
3. Log all invalid access attempt
4. Data audits

Authorization: Authorization is a security mechanism to determine user privileges on access levels related to system resources including computer programs, files etc. Authorization is normally proceeded by authentication for user identity verification.

- It is a critical aspect of database security and helps control who can perform various operations on the data stored in the database.

Types of authorization: 8 types.



1. Read authorization: Allows reading, but not modification of data.
2. Insert authorization: Allows insertion of new data, but not modification of existing data.
3. Update authorization: Allows modification, but not deletion of data.
4. Delete authorization: Allows deletion of data.
5. Index authorization: Allows creation and deletion of indices.
6. Resources authorization: Allows creation of new relations.
7. Alteration authorization: Allows addition or deletion of attributes in a relation.
8. Drop authorization: Allows deletion of relations.

Authorization and views

- ☐ Users can be given authorization on views, without being given any authorization on the relations used in the view definition.
- ☐ A combination on relational-level security and view-level security can be used to limit a user's access to precisely the data that user needs.

Authorization on Views

- ☐ Creation of view does not require resources authorization since no real relation is being created.
- ☐ If creator of view cust-loan had only read authorization on borrower and loan he gets only read authorization on cust loan.

Example: suppose a bank clerk needs to know the names of the customers of each branch, but is not authorized to see specific loan information.

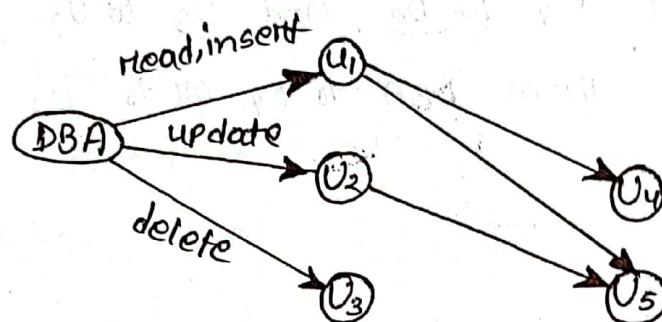
- ☐ The cust-loan view is defined in SQL as follows;

Create view cust-loan as

```
select branchname, customer-name  
from borrower, loan  
where borrower.loan-number = loan.loan-number
```

- ③ The clerk is authorized to see the result of the query:
- select*
from cust-loan

Authorization Grant Graph:



- The passage of authorization from one user to another maybe represented by an authorization graph.
- The nodes of this graph are the users.
- The root of the graph is the database administration.
- And edge $U_i \rightarrow U_j$ indicates that user U_i has granted update authorization on loan to U_j .

⇒ If DBA revokes grant from U_1 :

- Grant must be revoked from U_4 since U_1 no longer has authorization
- Grant must not be revoked from U_5 since U_5 has another authorization path from DBA through U_2

⇒ Must prevent cycles of grants with no path from the root:

- DBA grants authorization to U_7
- U_7 grants authorization to U_8
- U_8 grants authorization to U_7
- DBA revokes authorization from U_7

⇒ Must revoke grant U_7 to U_8 and U_8 to U_7 since there is no path from DBA to U_7 or to U_8 anymore.

□ The grant statement is used to confer authorization
grant <privilege list>
on <relation name or view name> to <user list>

<user list> is:

- a user id
- Public, which allows all valid users
- the privilege granted
- A role (more on this later)

Revoking authorization in SQL:

☐ The revoke statement is used to revoke authorization.

revoke <privilege list>

on <relation name or view name> from <user list> [restrict | cascade]

☐ Ex: revoke select on branch from U_1, U_2, U_3 cascade

☐ we can prevent cascading by specifying restrict:

revoke select on branch from U_1, U_2, U_3 restrict

with restrict, the revoke command fails if cascading revokes are required.

☐ <privilege-list> may be all to revoke all privileges the revokee may hold.

☐ If <revokee-list> includes public all users lose the privilege except those granted it explicitly.

☐ If the same privilege was granted twice to the same user by different grantees, the user may retain the privilege after the revocation

☐ All privileges that depend on the privilege being revoked are also revoked.

Limitations of SQL Authorization:

- SQL does not support authorization at a tuple level
- All end-users of an application may be mapped to a single database user.
- with the growth in web access to databases, database accesses come primarily from application servers.
- All end-users of an application may be mapped to a single database user.

Privileges in SQL:

☐ Select: Allows read access to relation.

Ex: grant users U_1 , U_2 and U_3 select authorization on the branch relation:

grant select on branch to U_1 , U_2 , U_3

☐ Insert: the ability to insert tuples

☐ update: the ability to update using the SQL update statement

☐ Delete: Ability to delete tuples.

☐ References: Ability to declare foreign keys when creating relations.

@ usage: Authorizes a user to use a specified domain.

@ All privileges: Used as a short form for all the allowable Privileges.

With grant option: Allows a user who is granted a privilege to pass the privilege on to other users.

Ex: Grant select branch to U₁ with grant option.
gives U₁ the select privileges on branch and allows U₁ to grant this privilege to others.

Role: A set of roles is created in the database. Authorizations can be granted to roles in exactly the same ~~function~~ fashion as they are granted to individual users. Each database user is granted a set of roles (which may be empty) that she is authorized to perform.

- Privileges can be granted or revoked from roles just like user
- Roles can be assigned to users, and even to other roles.

create role teller

create role manager

grant select on branch to teller

grant update | balance on account to teller

grant all privileges on account to manager

grant tellon to manager

grant tellon to alice, bob

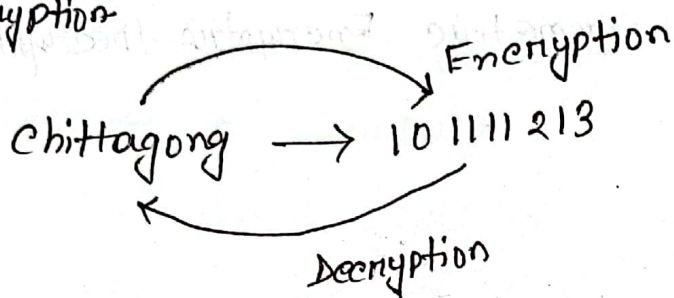
grant manager to avi

Encryption: Refers to the process of converting the data stored in a database. Data may be encrypted when database authorization do not offer sufficient Protection. It depends not on the secrecy of the algorithm.

- ❑ Data Encryption Standard (DES)
- ❑ Advanced Encryption Standard (AES): It is a new standard replacing DES, and is based on the Rijndael algorithm but is also dependent on shorter secret keys.
- ❑ Public key encryption: It is based on each user having two keys:
 - Public key: publicly published key used to encrypt data but cannot be used to decrypt data.
 - Private key: key known only to individual user and used to decrypt data.

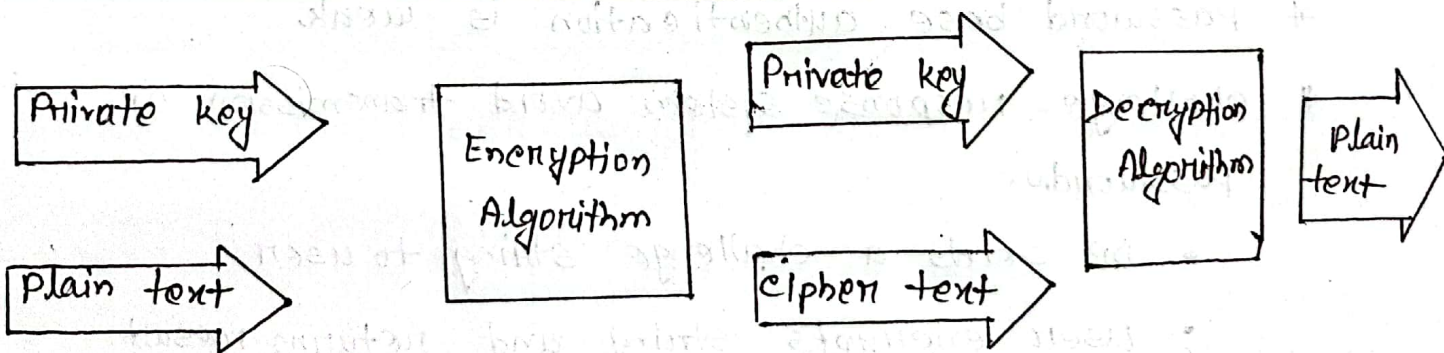
Decryption: It is the reverse process of encryption.
i.e. to gain back the original text/data from an encrypted cipher text.

A decryption key is a specific piece of information that is used to reverse the process of encryption.

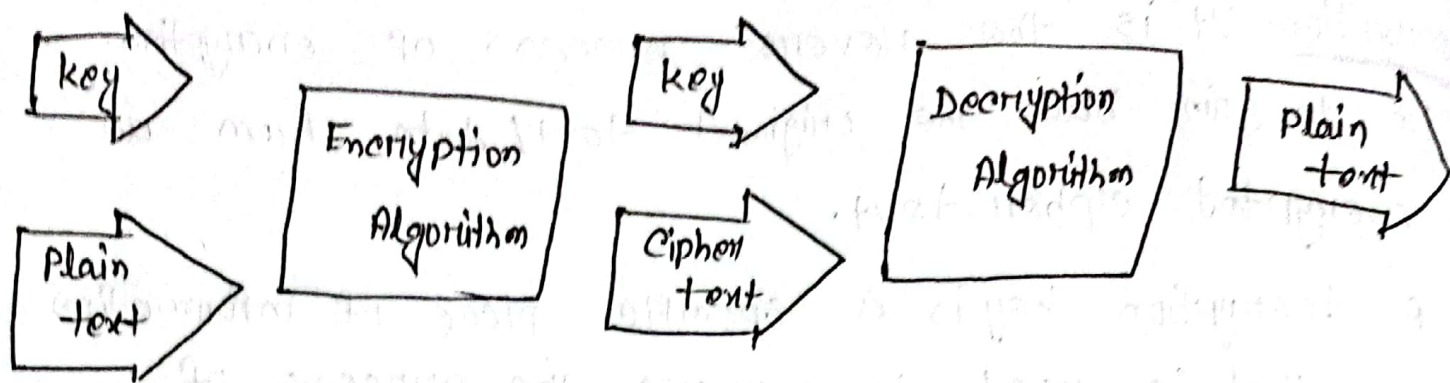


- ⇒ Symmetric key (same key)
- ⇒ Asymmetric key (Different key)
- ⇒ Private key
- ⇒ Public key

Steps of Encryption Decryption algorithm:



Flowchart of Asymmetric Encryption Decryption Process



Flow chart of symmetric Encryption/Decryption Process

Authentication: Database authentication is the process or act of confirming that a user who attempting to log into database is authorized to do so and is only accorded the rights to perform activities that one has been authorized to do.

Password base authentication is weak.

challenge-response system avoid transmission of passwords.

- DB sends a challenge string to user
- user encrypts string and returns result
- DB verifies identity by decrypting result
- Can use public key encryption system by DB sending a message.

Digital signatures are used to verify authenticity of data.

Audit trails: An audit trails is a log of all changes to the database along with information.

It is a record of all the activities and changes made to a database.

- ⇒ used to track erroneous updates.
- ⇒ Can be implemented using triggers, but many database systems provide direct support.